

Informační technologie

Rozšířené studijní materiály

Studijní materiál zahrnující deset stěžejních okruhů — od stavebních prvků počítače přes architekturu instrukční sady, přerušení, paralelismus a paměťovou hierarchii až po kybernetickou bezpečnost a obranu webových aplikací. Strukturovaný přehled je doplněn o detailní výklad, příklady a praktické souvislosti.

Architektura počítačů · Bezpečnost infrastruktury

Obsah

- 1. Základní stavební prvky počítače**
- 2. Architektura instrukční sady (ISA)**
- 3. Přerušování - princip, typy a obsluha**
- 4. Proudové zpracování instrukcí (pipelining), RISC vs. CISC**
- 5. Hierarchie paměťového subsystému a cache**
- 6. Paralelismus v počítačových architekturách**
- 7. Architektura pro monitorování, detekci a prevenci útoků**
- 8. Technická a organizační opatření pro bezpečnost**
- 9. Zabezpečení vnějšího perimetru a interních systémů**
- 10. Útoky na webovou infrastrukturu, dopady a obrana**

1. Základní stavební prvky počítače

Počítač je tvořen několika spolupracujícími subsystémy: procesorem, pamětí, vstupně-výstupními zařízeními, grafickou kartou a sběrnici. Tato kapitola popisuje jejich funkce, vzájemné vztahy i dvě hlavní architektonické filozofie — von Neumannovu a Harvardskou.

1.1 CPU - procesor

Procesor (CPU) je „mozek“ počítače — vykonává instrukce programů a řídí chod celého systému. Uvnitř CPU najdeme řadič (control unit), aritmeticko-logickou jednotku (ALU), pracovní registry a interní sběrnice.

- **ALU (sčítačka)** - aritmeticko-logická jednotka, která provádí základní aritmetické a logické operace nad daty (sčítání, odečítání, násobení, AND/OR/XOR, posuny, porovnávání). U výkonných CPU se k ní přidávají i další jednotky — FPU pro plovoucí řádovou čárku a vektorové (SIMD) jednotky.
- **Řadič** - načítá instrukce z paměti, dekoduje je a pomocí řídicích signálů koordinuje činnost ALU, registrů a vstupně-výstupních obvodů. Řadič tedy řídí tok dat mezi jednotlivými částmi počítače.
- **Registry** - velmi rychlá paměť uvnitř CPU pro dočasné uložení operandů a mezivýsledků. Jejich kapacita je malá (desítky až stovky bajtů), ale přístup trvá jen jeden takt.
- **Cache** - velmi rychlá vyrovnávací paměť uvnitř CPU pro často používaná data. Bývá hierarchická: **L1** je nejrychlejší a nejmenší (řádově desítky kB), **L2** větší a o trochu pomalejší, **L3** největší a sdílená mezi jádry. Číslo úrovně přibližně odpovídá vzdálenosti od jádra: čím nižší, tím rychlejší.

Souvislost

Moderní CPU obsahují i další funkční bloky — jednotku pro plovoucí řádovou čárku (FPU), vektorové instrukce (SSE, AVX, NEON) a v poslední době i akcelerátory pro neuronové sítě. Vše se spojuje vnitřní sběrnici, která propojuje řadič, ALU a registry.

1.2 Paměť

Paměť slouží k uchování dat a instrukcí, se kterými CPU pracuje. Z hlediska funkčního i fyzického rozeznáváme několik druhů, které tvoří paměťovou hierarchii (viz kapitola 5).

- **RAM (operační paměť)** - dočasná paměť pro data a instrukce běžících programů. Realizuje se jako DRAM o kapacitách v jednotkách až desítkách GB. Je rozdělena na buňky stejné velikosti a každá má unikátní adresu.
- **ROM (read-only)** - obsahuje firmware (BIOS/UEFI) a základní software nezbytný pro spuštění systému. V harvardských architekturách v ní bývá uložen i samotný program (proto se nemůže modifikovat).
- **Cache** - velmi rychlá paměť mezi CPU a RAM, držící kopie naposledy používaných dat. Cache hit je velmi rychlý, cache miss vede k dotazu do pomalejší úrovně.

- **Virtuální paměť** – rozšíření RAM o část úložiště (SSD/HDD) skrze stránkování. OS přemísťuje neaktivní stránky paměti na disk a zpět; tím dovolí běh programů, které by se do RAM nevešly.

1.3 Periferie

Periferie jsou externí zařízení, která umožňují počítači interakci s okolím. Komunikují s CPU typicky přes sběrnice a řadiče a jejich operace jsou často **asynchronní** — řešené pomocí přerušení (viz kapitola 3).

- **Vstupní zařízení:** klávesnice, myš, mikrofon, skener, kamera.
- **Výstupní zařízení:** monitor, reproduktory, tiskárna.
- **Vstupně-výstupní zařízení:** dotykový displej, síťová karta, USB rozhraní, externí disky.

1.4 GPU - grafická karta

GPU je specializovaný procesor pro akceleraci grafických výpočtů a obecně silně paralelních úloh. Obsahuje stovky až tisíce paralelních výpočetních jader, která efektivně provádějí stejnou operaci nad obrovským počtem dat (matice pixelů, vrcholy, tenzory neuronové sítě).

	CPU	GPU
Počet jader	Jednotky až desítky výkonných jader	Stovky až tisíce jednodušších jader
Typická úloha	Sekvenční úlohy, větvení, řízení	Paralelní výpočty se stejnou strukturou
Optimalizace	Latence jedné instrukce	Propustnost (mnoho operací zároveň)
Vlastní paměť	Sdílí RAM	VRAM (např. GDDR6, HBM)

Moderně se GPU nepoužívá jen k vykreslování — výrazně se uplatňuje při trénování neuronových sítí, simulacích, kryptoměnách a dalších paralelních výpočtech (technologie CUDA, OpenCL, ROCm).

1.5 Sběrnice

Sběrnice propojují části počítače a umožňují jim komunikovat. Tvoří je soustava vodičů pro přenos dat, adres a řídicích signálů. Mají definovanou šířku (počet bitů přenesených najednou) a komunikační protokol.

- **Datová sběrnice** – přenáší samotná data mezi komponentami.
- **Adresová sběrnice** – nese adresy paměťových buněk či vstupně-výstupních portů, kam se data zapisují nebo odkud se čtou.
- **Řídicí sběrnice** – přenáší řídicí signály (čtení/zápis, potvrzení, přerušení), které synchronizují činnost komponent.

Příklady moderních sběrnic: PCI Express (grafická karta, NVMe SSD), SATA (klasické disky), USB (univerzální vnější sběrnice), NVMe (rozhraní pro rychlé SSD nad PCIe). Vnitřní sběrnice CPU propojují ALU, registry a řadič.

1.6 Von Neumannova vs. Harvardská architektura

Dvě klasické architektonické filozofie rozhodují o tom, zda se instrukce a data dělí o stejnou paměť, nebo zda mají paměti oddělené.

Von Neumannova architektura

- Paměť pro instrukce i data je společná — sdílejí jednu paměťovou sběrnici.
- Jednoduchost a flexibilita: program i data lze ukládat stejným způsobem; je možná tzv. samomodifikace kódu.
- Nevýhoda — **von Neumannovo úzké hrdlo**: instrukce a data se nemohou načítat současně, což omezuje propustnost.
- Typická pro většinu běžných PC.

Harvardská architektura

- Instrukce a data mají oddělené paměti i sběrnice.
- Paralelní přístup umožňuje vyšší výkon — CPU může v jednom taktu načítat instrukci a operand současně.
- Často v mikrokontrolérech, DSP (digitální signálové procesory), ARM jádrech a vestavěných systémech.
- Moderní procesory běžně používají **modifikovanou Harvardskou architekturu**: navenek působí jako von Neumann, ale L1 cache jsou oddělené pro instrukce a data (split cache).

1.7 Instrukční cyklus počítače

Aby procesor program vykonal, opakovaně provádí **instrukční cyklus** — sled kroků potřebných ke zpracování jedné strojové instrukce. Cyklus se opakuje, dokud běží počítač.

1. **Fetch (načtení)** – řadič načte z paměti instrukci podle adresy uložené v **programovém čítači (PC)**. Instrukce se přenese po datové sběrnici do **instrukčního registru (IR)**.
2. **Decode (dekódování)** – řadič rozšifruje binární kód v IR, určí typ operace, operandy a připraví řídicí signály.
3. **Execute (provedení)** – ALU nebo jiná jednotka provede požadovanou operaci (výpočet, přesun dat, skok).
4. **Memory Access** – pokud je třeba, instrukce čte nebo zapisuje data do paměti.
5. **Writeback** – výsledek se uloží zpět do registru nebo paměti.

V klasickém ne-pipelinovaném CPU trvá jedna instrukce řádově 5 taktů (po jedné fázi za takt). Moderní procesory tyto fáze **překrývají** — viz kapitola 4 (pipelining) — a mohou tak teoreticky dokončit jednu instrukci za jeden takt.

2. Architektura instrukční sady (ISA)

ISA je abstraktní rozhraní mezi hardwarem a softwarem — definuje „jazyk“, kterému procesor rozumí. Bez ISA by překladač nevěděl, jaké instrukce má vygenerovat. Procesory se stejnou ISA jsou binárně kompatibilní (např. Intel a AMD pro x86), i když se jejich vnitřní mikroarchitektura liší.

2.1 Co ISA zahrnuje

Instruction Set Architecture (ISA) je souhrn všech programovatelných vlastností procesoru — to, co vidí programátor a překladač. Patří sem:

- Soubor instrukcí, které procesor podporuje (opcode i sémantika).
- Model registrů, datové typy a způsoby adresování operandů.
- Formáty instrukcí (jejich kódování v binární podobě).
- Systém přerušení a řízení toku programu (skoky, volání podprogramů).
- Privilegované instrukce, paměťový model, pravidla vstupu/výstupu.

2.2 Typy instrukčních sad

CISC (Complex Instruction Set Computing)

- Komplexní instrukce, které dokáží vykonat více kroků (např. řetězení, přímé násobení). Cílem bylo přiblížit strojové instrukce vyšším jazykům.
- Délka instrukcí je **variabilní** (1–15 bajtů u x86), formátů je mnoho a dekodování je složitější.
- Větší počet adresovacích režimů, méně registrů, použití mikroprogramového řízení.
- Použití: x86 (Intel, AMD), VAX, IBM System/360.

RISC (Reduced Instruction Set Computing)

- Malá sada jednoduchých instrukcí, většinou pevné délky (např. 32 b). Většina instrukcí se vykoná za jeden takt.
- Architektura **load/store**: do paměti přistupují pouze instrukce LOAD/STORE; ostatní operace pracují čistě s registry.
- Hodně registrů, jednoduché dekodování, ideální pro pipelining.
- Použití: ARM, MIPS, PowerPC, RISC-V.

VLIW (Very Long Instruction Word)

- Jedno velmi široké instrukční slovo obsahuje několik nezávislých operací určených pro paralelní jednotky. Paralelismus rozhoduje **překladač**, hardware je jednoduchý.
- Použití: DSP procesory, raně i superpočítače, Intel Itanium (varianta EPIC).

EPIC (Explicitly Parallel Instruction Computing)

- Vychází z VLIW, ale s lepší správou paralelních operací — explicitní informace o závislostech v kódu pomáhají hardwaru dynamicky plánovat.
- Použití: Intel Itanium (architektura IA-64).

Shrnutí

RISC = málo jednoduchých instrukcí (rychlé, jednotné, ale je jich potřeba víc).
 CISC = bohatá sada komplexních instrukcí (silné jednotlivě, ale složitější HW).
 Dnes se hranice stírá: x86 procesory uvnitř dekódují CISC instrukce do jednodušších *mikrooperací* RISC-stylu, takže rozdíl je dnes spíše v ISA než v mikroarchitektuře.

2.3 Typické skupiny instrukcí

Skupina	Popis	Příklady
Aritmeticko-logické	Počítání, porovnávání, bitové operace	ADD, SUB, AND, OR, XOR, CMP
Datový přesun	Přesuny mezi registry a paměti; práce se zásobníkem	MOV, LOAD, STORE, PUSH, POP
Řídící (skokové)	Změna toku programu — podmíněné/nepodmíněné skoky, volání	JMP, JZ, JNZ, CALL, RET
Speciální / systémové	Přerušování, nastavení registrů, synchronizace	INT, HLT, NOP, IRET
Plovoucí desetinná čárka	Výpočty s reálnými čísly (FPU, SIMD)	FADD, FSUB, FMUL, FDIV

2.4 Adresní módy

Adresní (adresovací) režim určuje, jak instrukce získá svůj operand. Bohatství adresovacích módů odlišuje CISC (mnoho variant) od RISC (typicky jen pár základních).

Mód	Význam	Příklad
Přímý (immediate)	Hodnota operandu je přímo v instrukci	MOV AX, 5
Přímé adresování	Operand je v paměti na uvedené adrese	LOAD R1, (1000h)
Nepřímý	Adresa operandu je uložena v registru	MOV AX, [BX]
Registrový	Operand je přímo v registru	ADD R1, R2
Indexovaný (bázový)	Adresa = registr + offset (užitečné pro pole)	MOV AX, [BX + SI]
Relativní	Adresa se počítá relativně vůči aktuální pozici (PC)	JMP +5
Zásobníkový	Implicitní, používá vrchol zásobníku (SP)	POP R1

2.5 Kódování (formát) instrukcí

Strojový kód má pole pro **opcode** (kód operace) a další pole pro operandy a adresovací režimy.

- **RISC - pevná délka:** např. MIPS má všechny instrukce 32-bitové. Formát R-type: opcode(6) | rs(5) | rt(5) | rd(5) | shamt(5) | funct(6). Jednotná délka výrazně zjednodušuje pipeline.

- **CISC - proměnlivá délka:** u x86 může mít instrukce 1–15 B (s prefixy, ModR/M bytem, SIB, displacementem, immediate). Dekódování je složitější, ale program v paměti bývá kompaktnější.
- **Speciální formáty:** ARM Thumb-2 míchá 16- a 32-bitové instrukce, VLIW kombinuje více opcode v jednom dlouhém slovu.

3. Přerušení - princip, typy a obsluha

Přerušení (interrupt) umožňuje procesoru reagovat na události bez neustálého dotazování zařízení (tzv. polling). Místo toho zařízení samo signalizuje, že potřebuje obsluhu, a CPU dočasně přeruší běžný program a přejde do servisní rutiny.

3.1 Princip přerušení

Přerušení lze chápat jako neplánované „volání podprogramu“ vyvolané externí událostí nebo vnitřní chybou. Po obsluze se procesor vrací na místo, kde program přerušil.

- Procesor uloží aktuální stav (registry, programový čítač).
- Skočí na obslužný podprogram (**Interrupt Service Routine — ISR**).
- Po skončení ISR se vrací zpět a pokračuje v původním běhu.

3.2 Porovnání s voláním podprogramu

Volání podprogramu i přerušení se v mnohém podobají, ale existují důležité rozdíly.

	Volání podprogramu	Přerušení
Vyvolání	Synchronně - instrukcí CALL v kódu	Asynchronně - HW signál nebo výjimka
Zdroj	Programátor / běžící program	Hardware, OS nebo speciální instrukce (INT)
Ukládaný stav	Návratová adresa, případně volaným uložené registry	Návratová adresa + stavové slovo, často část registrů
Priorita / vnořování	Dáno logikou programu	Priority HW, vyšší může přerušit nižší
Návrat	Instrukce RET	Speciální IRET, obnoví celý kontext

Společné: obě situace způsobují dočasný skok na jinou část kódu a v obou se ukládá návratová adresa.

3.3 Typy přerušení

- **Hardwarová (vnější) přerušení** - přicházejí z periférií (klávesnice, disk, časovač, síťová karta). Jsou *asynchronní* — mohou nastat kdykoli.
- **Softwarová přerušení** - vyvolává je program speciální instrukcí (např. INT n v x86). Slouží typicky pro systémová volání jádra OS.
- **Vnitřní (výjimky / traps)** - generuje sám CPU při chybě: dělení nulou, neplatná instrukce, page fault, porušení ochrany paměti. Jsou *synchronní* s vykonávanou instrukcí.
- **Maskovatelná vs. nemaskovatelná (NMI)** - maskovatelná lze dočasně blokovat příznakem v procesoru; NMI je vždy zpracováno (kritická událost — selhání napájení, paměti).

3.4 Mechanismus obsluhy přerušení

1. Zařízení nebo CPU vygeneruje přerušovací signál.
2. Procesor dokončí **právě běžící instrukci** (přerušení se testuje mezi instrukcemi, aby nebyla narušena atomicita).
3. Uloží **kontext**: stav registrů, programový čítač, případně stavové slovo. Část ukládá HW automaticky, zbytek pak ISR.
4. Načte adresu ISR z **přerušovací tabulky (Interrupt Vector Table)**. Vektor je číslo, které se získá od HW řadiče a indexuje tabulku.
5. Proveďte skok na **ISR (Interrupt Service Routine)** a vykoná obslužný kód — přečte data z bufferu, zpracuje je, resetuje stav periférie. Během obsluhy bývají další přerušení dočasně zakázána.
6. Po dokončení obsluhy se návrat provádí instrukcí IRET, která obnoví uložený kontext, takže přerušený program pokračuje od místa, kde skončil.

3.5 Zdroje přerušení

- **Vstupní zařízení**: klávesnice, myš, dotykový displej.
- **Časovač / hodiny**: periodická přerušení pro plánovač OS, RTC.
- **Síťová zařízení**: oznámení o příchodu paketu.
- **Diskové / paměťové operace**: dokončení I/O, DMA přenos.
- **Chyby a výjimky CPU**: dělení nulou, chyba ochrany paměti, neplatná instrukce, page fault.

3.6 Výběr přerušení k obsluze a IVT

Pokud o pozornost CPU žádá více zařízení současně, rozhoduje řadič přerušení (např. PIC nebo modernější APIC):

- Řadič shromažďuje žádosti, určuje **prioritu** a vybere tu nejvyšší.
- CPU od řadiče dostane **číselný vektor**, kterým indexuje tabulku.
- **Interrupt Vector Table (IVT)** obsahuje adresy ISR pro každé přerušení.

Příklad — x86 reálný režim

IVT začíná na adrese 0x0000:0000; vektor 0 zaujímá bajty 0-3, vektor 1 bajty 4-7 atd. Procesor pomocí vektoru určí počáteční adresu obslužné rutiny a skokem na ni začne ISR.

4. Proudové zpracování instrukcí (pipelining)

Pipelining je klíčová technika zvyšování výkonu. Rozdělí provádění instrukce do několika fází, které se překrývají — podobně jako na výrobní lince. Tato kapitola také srovnává RISC a CISC procesory a popisuje hazardy, které mohou pipeline narušit.

4.1 Rozdíly mezi RISC a CISC procesory

RISC (Reduced Instruction Set Computer)

- Malá, jednoduchá a optimalizovaná sada instrukcí.
- Pevná délka instrukcí (typicky 32 b), jednotný formát.
- Většina instrukcí se vykoná za **1 takt** (ideálně).
- Silná orientace na operace registr-registr; paměť se používá jen pro LOAD/STORE (load/store architektura).
- Velké množství registrů (typicky 32 obecných registrů).
- Vhodné pro mobilní zařízení a vestavěné systémy díky efektivnímu poměru výkon/spotřeba (ARM, MIPS, PowerPC, RISC-V).

CISC (Complex Instruction Set Computer)

- Rozsáhlá a komplexní sada instrukcí, často s různou délkou.
- Jedna instrukce může provést více operací (např. načtení z paměti, výpočet a uložení v jednom kroku).
- Některé instrukce trvají více hodinových cyklů (násobení, dělení, složité přístupy do paměti).
- Snadnější programování v assembleru — bohaté adresovací režimy.
- Použito v PC a serverech: x86 (Intel, AMD), VAX, IBM System/360.

Vlastnost	RISC	CISC
Velikost instrukční sady	Malá	Velká
Délka instrukcí	Konstantní	Variabilní
Rychlost provádění	Vysoká (často 1 takt)	Nižší (více taktů)
Počet cyklů na instrukci	Většinou 1 cyklus	Více cyklů
Dekódování instrukcí	Jednoduché a rychlé	Komplikovanější a pomalejší
Použití registrů	Mnoho registrů	Více operací s pamětí
Složitost instrukcí	Jednoduché	Komplexní
Efektivita HW	Vysoká (jednodušší čip)	Nižší (složitější dekodér)
Typické použití	Mobilní zařízení, embedded	PC, servery

Stírání hranic

Moderní x86 procesory uvnitř dekodují CISC instrukce na řadu jednoduchých *mikrooperací* RISC-stylu, které pak prochází pipeline. Naopak ARMy převzaly mnoho technik dříve typických pro CISC (out-of-order, predikce skoků, superskalární vydávání).

4.2 Princip proudového zpracování (pipelining)

Provádění instrukce se rozdělí do několika za sebou jdoucích fází, které pracují **souběžně** nad různými instrukcemi. Zatímco jedna instrukce se dekoduje, jiná se už načítá a další provádí. Procesor tak v každém taktu pokročí všechny rozpracované instrukce o jednu fázi.

Typické fáze 5-stupňové pipeline:

1. **IF (Instruction Fetch):** načtení instrukce z paměti.
2. **ID (Instruction Decode):** dekodování a načtení operandů.
3. **EX (Execute):** provedení operace v ALU.
4. **MEM (Memory Access):** přístup do paměti, je-li potřeba.
5. **WB (Write Back):** zápis výsledku zpět do registru.

4.3 Jak pipelining zvyšuje výkon

- Umožňuje teoreticky **dokončit jednu instrukci za takt** (po naplnění).
- Místo čekání na dokončení celé instrukce se paralelně zpracovává více instrukcí.
- Efektivně využije všechny části procesoru (ALU, dekodér, paměť).
- Snižuje zpoždění způsobené jednotlivými fázemi zpracování.

Numerický příklad

Bez pipelingu by 5 instrukcí trvalo $5 \times 5 = 25$ taktů (každá má 5 fází). S pipeliningem trvá první instrukce 5 taktů, ale každá další jen 1 takt — celkem **9 taktů**. Při velkém počtu instrukcí se průměr blíží 1 instrukci na takt.

4.4 Co může pipeline narušit (hazardy)

Hazardy jsou překážky, které brání plynulému průchodu instrukcí pipeline:

- **Datové hazardy:** instrukce potřebuje výsledek předchozí, který ještě není zapsán. Např. ADD R1,R2 produkuje R1 ve fázi WB, ale následující SUB R3,R1 by jej potřebovala už v EX.
- **Řídicí hazardy:** u skoků a větvení není dopředu znám cíl. Pipeline mohla mezitím spekulativně načíst nesprávné instrukce, které je pak nutné zahodit (flush).
- **Strukturální hazardy:** dvě instrukce potřebují stejný HW prostředek (např. ALU nebo paměťový port) ve stejném taktu.

Řešení hazardů:

- **Forwarding (přeposílání):** výsledek se předá přímo z výstupu ALU na vstup další ALU, aniž by čekal na zápis do registru.

- **Stalls / vkládání bublin:** CPU dočasně pozastaví pipeline, dokud nejsou data k dispozici.
- **Prediktory větvení:** odhadnou, kterou cestou se program vydá, a pipeline se plní spekulativně. Pokud predikce selže, dochází k drahému flush a ztrátě několika taktů.
- **Oddělení paměti pro instrukce a data (Harvardský přístup):** odstraňuje strukturální hazardy mezi fetch a memory access.

4.5 RISC v mobilních zařízeních

V mobilech (chytrých telefonech, tabletech) dnes dominují procesory **ARM**, případně **RISC-V**. Důvodem je skvělý poměr výkon na watt — RISC architektura má jednodušší design, méně tranzistorů věnovaných složité logice, menší spotřebu a nižší zahřívání.

- **big.LITTLE architektura:** kombinace výkonných a úsporných jader na jednom čipu. Nenáročné úkoly běží na úsporných jádrech.
- **SoC (System on Chip):** ARM jádra se snadno integrují s GPU, DSP, modemem, AI akcelerátory na jednom čipu.
- **Pokročilé mikroarchitektury:** moderní ARMy jsou superskalární, out-of-order, s prediktory větvení — výkonem dnes konkurují x86 (viz Apple M-series).

5. Hierarchie paměťového subsystému a cache

Paměťový subsystém kombinuje rychlost, kapacitu a cenu prostřednictvím hierarchie pamětí. Cache mezi CPU a RAM zásadně ovlivňuje výkon — překlenuje obrovský rozdíl rychlostí mezi procesorem a hlavní pamětí (tzv. memory wall).

5.1 Struktura a funkce jednotlivých komponent

Hierarchie je navržena tak, aby procesor měl co nejčastěji potřebná data v rychlé paměti. Vyšší úroveň hierarchie je rychlejší, ale menší a dražší (cena za výkon).

Úroveň	Rychlost	Kapacita	Funkce
Registry CPU	1 takt	desítky až stovky B	Operandy aktuální instrukce
L1 cache	několik taktů	desítky kB	Nedávno použitá data a instrukce
L2 cache	~10 taktů	stovky kB až jednotky MB	Větší vyrovnávací paměť
L3 cache	~30 taktů	jednotky až desítky MB	Sdílená mezi jádry
RAM (operační paměť)	~100–300 taktů	GB	Hlavní pracovní paměť programů
Disk (SSD/HDD)	tisíce až milion taktů	GB–TB	Trvalé úložiště
Cloud / síťové úložiště	nejpomalejší	téměř neomezená	Archiv, sdílení, zálohy

5.2 Vliv cache na výkon počítače

Cache výrazně zlepšuje výkon tím, že drží nejčastěji používaná data blízko procesoru. CPU tak nemusí často sahat do pomalé RAM.

- **Cache hit** – data jsou v cache; přístup je rychlý (jednotky taktů).
- **Cache miss** – data nejsou v cache, je nutné je načíst z nižší úrovně.

Čím vyšší **hit rate** (typicky 95–99 % u L1+L2), tím lepší výkon. Bez cache by procesor většinu času čekal na paměť — rozdíl rychlostí CPU a RAM je obrovský.

5.3 Časová a prostorová lokalita

Cache funguje efektivně díky typickému chování programů — vykazují **lokalitu odkazů**:

- **Časová lokalita:** nedávno použitá data se pravděpodobně brzy použijí znovu (např. proměnná uvnitř smyčky). Cache nedávno použité hodnoty ponechává.
- **Prostorová lokalita:** pokud program přistupuje k určité adrese, často brzy přistoupí i k adresám sousedním (sekvenční průchod polem). Proto cache načítá celý **blok dat (cache line)**, typicky 32–128 B, nejen jedno slovo.

5.4 Konstrukce cache paměti

Cache je rozdělena do **bloků (cache lines)**. Při přístupu CPU porovná adresu s tagy a zjistí, zda požadovaná adresa odpovídá nějakému bloku. Mapování dat do cache (její **asociativita**) má tři základní varianty:

- **Přímé mapování (direct-mapped):** každý paměťový blok může být uložen jen na jedno místo. Velmi jednoduché, ale dochází ke konfliktům — bloky se stejným indexem se vzájemně vytlačují.
- **Set-associative cache (N-cestná):** kompromis. Cache je rozdělená do sad po N pozicích; blok mapuje do konkrétní sady, ale v rámci sady může být kdekoli. Typicky 2–8cestná. Nižší konflikty než direct-mapped.
- **Plně asociativní:** libovolná adresa může být uložena kdekoli v cache. Maximální flexibilita, ale velmi drahé hledání (porovnání se všemi tagy paralelně). Použití jen pro malé cache (TLB).

Politika výměny: když je cache plná, je třeba něco vyhodit. Nejčastější algoritmus je **LRU (Least Recently Used)** — vyhodí se nejdéle nepoužitý řádek. Používají se i jednodušší heuristiky (random, FIFO, pseudo-LRU).

Politika zápisu: write-through (zápis okamžitě i do RAM — jednodušší koherence, ale pomalejší) vs. **write-back** (do RAM se zapíše až při vyhození z cache, řádek je označen jako „dirty“). Většina moderních CPU používá write-back u L1/L2.

5.5 Nižší vs. vyšší stupeň asociativity

- **Nižší asociativita:** jednodušší implementace, rychlejší přístup, ale více konfliktů (data si „překážejí“).
- **Vyšší asociativita:** méně konfliktů, vyšší šance na hit, ale složitější hardware a pomalejší hledání tagů.

V praxi se obvykle volí kompromis (např. 4-cestně asociativní L1).

5.6 Typy výpadků (cache miss)

- **Compulsory miss (cold miss):** první přístup k danému bloku — ještě nikdy nebyl v cache.
- **Capacity miss:** cache je příliš malá, aby udržela všechna potřebná data.
- **Conflict miss:** data se „bijí“ o stejnou pozici v cache kvůli způsobu mapování (typicky u direct-mapped).

5.7 Prefetching a optimalizace přístupů

Prefetching je technika, kdy se data načítají do cache *předtím*, než je program skutečně potřebuje. Tím se skryje paměťová latence.

- **Hardwarový prefetch:** CPU monitoruje přístupové vzory (např. sekvenční čtení) a automaticky načítá další bloky dopředu.
- **Softwarový prefetch:** programátor nebo překladač vloží prefetch instrukci (PREFETCH v x86, PLD v ARM), pokud ví, že data budou brzy potřeba.

Doporučení pro programátory

Pište kód s ohledem na lokalitu — sekvenční průchod polem je dramaticky rychlejší než náhodný skok po velkých vzdálenostech (cache miss). Vyhýbejte se „pointer chasing“ ve velkých datových strukturách, pokud je výkon kritický.

6. Paralelismus v počítačových architekturách

Paralelismus zvyšuje výkon prováděním více operací současně. Existuje na několika úrovních — od ILP (paralelismu instrukcí v jednom vlákne) po distribuovaný paralelismus napříč počítači.

6.1 Typy paralelismu

- **ILP (Instruction-Level Parallelism):** více instrukcí jednoho vlákna se provádí najednou — pipelining, superskalární vydávání, out-of-order vykonávání.
- **TLP (Thread-Level Parallelism):** vlákna mohou běžet paralelně na různých jádrech, nebo sdílet jádro pomocí SMT (Hyper-Threading).
- **PLP (Process-Level Parallelism):** více nezávislých procesů běží současně na různých jádrech.
- **Distribuovaný paralelismus:** výpočty se dělí mezi více počítačů propojených sítí (HPC clustery, grid computing).

6.2.1 Skalární procesory

- Vykonávají jednu instrukci nad jedním datovým operandem v každém taktu.
- Žádný nebo minimální paralelismus — maximálně jednoduchý pipelining.
- Používají se v jednodušších nebo starších systémech.

6.2.2 Superskalární procesory

- Více paralelních výkonných jednotek umožňuje vydat **více než jednu instrukci za takt**.
- Hardwarový plánovač dynamicky vyhledává nezávislé instrukce a řídí jejich paralelní provádění.
- Příkladem jsou moderní procesory Intel a AMD (x86-64) — dokáží vydat 4 a více mikrooperací za cyklus.
- Často kombinováno s **out-of-order execution** — pokud je instrukce blokována, CPU vykoná mezitím jinou nezávislou.

6.2.3 Vícevláknové procesory (Multithreaded CPUs)

- Zvyšují využití výpočetních jednotek přepínáním mezi vlákny.
- **SMT (Simultaneous Multithreading):** více vláken běží současně na jednom jádře, sdílí výkonné jednotky, ale mají vlastní registry.
- Příklad: Intel Hyper-Threading — dvě logická jádra na jednom fyzickém. V ideálním případě nárůst propustnosti o 20–30 %.

6.2.4 Vícejádrové procesory (Multi-core CPUs)

- Procesor obsahuje více **plně samostatných jader**, každé s vlastní pipeline a L1/L2 cache.
- Paralelizace se realizuje na úrovni OS a aplikačního softwaru.
- Typické pro běžné PC, servery i mobilní zařízení.
- Důležitý je **cache koherenční protokol** (např. MESI), který zajistí, že jádra vidí konzistentní data.

6.2.5 VLIW procesory

- Jedna velmi dlouhá instrukce obsahuje více nezávislých operací, které se provádějí paralelně v různých jednotkách.
- Paralelizaci provádí **překladač** — ten analyzuje závislosti a sestavuje instrukce do dlouhých slov.
- Jednodušší hardware (žádný dynamický plánovač), ale vysoké nároky na optimalizaci kompilátoru. Špatná zpětná kompatibilita: jiná konfigurace jednotek vyžaduje nové překládání.
- Použití: DSP, GPU shadery, Intel Itanium (EPIC).

6.3 Kdo provádí paralelizaci

Typ procesoru	Kdo paralelizuje
Skalární	Žádná paralelizace — sekvenční provádění
Superskalární	Hardware (řadič) dynamicky plánuje instrukce
Vícevláknové (SMT)	Hardware (vláknový plánovač) + OS (alokace vláken)
Vícejádrové	Operační systém a programátor (OpenMP, pthreads...)
VLIW	Překladač (statické plánování při překladač)

6.4 Vliv optimalizace kódu překladačem

Optimalizace překladačem má zásadní vliv na výkon — zejména u architektur se statickým plánováním (VLIW), ale i u superskalárních a vícevláknových.

- **Odhalení paralelismu:** kompilátor analyzuje závislosti mezi instrukcemi a přeuspořádává je tak, aby šly provádět paralelně.
- **Optimalizace cyklů:** unrolling smyček, inlining funkcí, odstranění mrtvého kódu.
- **Vektorizace:** překladač převede skalární operace na SIMD instrukce (SSE, AVX, NEON).
- **Přednačítání dat (prefetch):** snižuje čekání na paměť.
- **U VLIW:** bez kvalitního překladače procesor nemůže efektivně využít své jednotky — kompilátor je „mozkem“ paralelizace.

Shrnutí

Optimalizovaný překladač je zásadní zejména u VLIW, ale pomáhá odemknout výkon i u superskalárních a vícevláknových architektur. Překladač rozhoduje, jak dobře hardware využije svůj paralelní potenciál.

7. Architektura pro monitorování, detekci a prevenci útoků

Moderní podniková infrastruktura by měla být chráněna vícevrstvou architekturou (defense-in-depth). Cílem je kombinovat preventivní, detekční a reakční opatření tak, aby selhání jedné vrstvy nebylo fatální.

7.1 Perimetrální ochrana

- **Stateful / Next-Generation Firewall (NGFW)** – filtruje příchozí i odchozí provoz podle stavu spojení a aplikačních charakteristik. Pracuje na principu *implicit deny*: vše, co není výslovně povolené, je blokováno. NGFW navíc kontroluje provoz i na aplikační vrstvě (HTTP, DNS) a může obsahovat IDS/IPS.
- **IDS / IPS (Intrusion Detection / Prevention Systems)** – IDS monitoruje provoz a detekuje vzorce útoků nebo anomálie a varuje. IPS jde dále — útok rovnou zablokuje. Využívají signatur i heuristik (behaviorální detekce).
- **VPN** – zabezpečené tunely (IPsec, SSL/TLS) pro vzdálený přístup zaměstnanců a partnerů. Šifrují komunikaci a ověřují identitu.
- **HoneyPot / HoneyNet** – izolované systémy nastražené jako návnada. Lákají útočníky do kontrolovaného prostředí, kde lze sledovat jejich techniky a včas je odhalit.
- **DMZ (demilitarizovaná zóna)** – oddělená síť pro veřejně dostupné servery (web, mail, VPN gateway). I při kompromitaci serveru v DMZ se útočník nedostane přímo do interní sítě.

7.2 Ochrana a segmentace interní sítě

- **Síťová segmentace** (VLAN, mikrosegmentace) – oddělení kritických systémů (databáze, OT) od běžných zón. Mezi segmenty platí pravidla interních firewallů. Útočník po průniku není automaticky všude — musí překonávat další bariéry.
- **NAC (Network Access Control)** – povolí přístup k síti pouze zařízením, která splňují bezpečnostní politiky (aktualizace, antivirus, šifrování).
- **SIEM (Security Information & Event Management)** – centrálně shromažďuje logy z firewallů, serverů, IDS/IPS a aplikací; v reálném čase je **koreluje** a hlásí anomálie. Pomáhá odhalit incidenty, které by při izolovaném pohledu unikly pozornosti.

7.3 Ochrana koncových zařízení (Endpoint Security)

- **EDR / XDR** – pokročilá detekce a reakce na hrozby na úrovni OS. Sleduje běh procesů, registry, soubory; izoluje napadené zařízení.
- **Antivirové a anti-malware technologie** – včetně ochrany proti ransomwaru a behaviorální analýzy.
- **Šifrování disků** (BitLocker, LUKS, FileVault) – chrání data při krádeži zařízení.
- **Patch management** – pravidelné záplatování OS, aplikací i firmware minimalizuje exploitaci známých zranitelností.

7.4 Ochrana aplikací a dat (aplikační vrstva)

- **WAF (Web Application Firewall)** – chrání webové aplikace a API před OWASP Top 10 (SQLi, XSS, CSRF) a aplikačními DDoS útoky.
- **RASP (Runtime Application Self-Protection)** – monitoruje běžící proces aplikace a aktivně zasahuje při zjištění hrozby.
- **Bezpečný SDLC** – SAST, DAST, IAST a pravidelné penetrační testy začleněné do DevSecOps pipeline.
- **Validace vstupů a parametrizované dotazy** – prevence SQL/NoSQL injection a command injection.
- **Bezpečné HTTP hlavičky** – HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy.
- **API Gateway** – centrální bod pro autentizaci, rate-limiting a ochranu mikroslužeb.
- **DLP (Data Loss Prevention)** – sleduje a blokuje neoprávněný únik citlivých dat (osobní údaje, obchodní tajemství) v klidu i přenosu.

7.5 Identita a přístupová kontrola

- **MFA** – multi-faktorová autentizace (TOTP, FIDO2, biometrie).
- **RBAC a ABAC** – řízení oprávnění podle role nebo atributů uživatele.
- **SSO a federace identit** (SAML 2.0, OIDC) – jednotné přihlášení, centralizovaná správa účtů.
- **PAM (Privileged Access Management)** – speciální správa administrátorských účtů, just-in-time přístup, nahrávání relací.

7.6 Bezpečnostní politika a řízení rizik

- Soulad s **ISO 27001**, **NIST CSF**, **GDPR**, **NIS2** a odvětvovými regulacemi.
- Pravidelné audity, risk assessment, penetrační testy.
- Školení zaměstnanců (security awareness, phishing simulace).

7.7 Monitoring, detekce a reakce

SIEM & SOAR

- Centralizovaný sběr logů (firewally, OS, cloud, SaaS) a korelace událostí.
- **SOAR** – automatizace reakce: izolace zařízení, reset hesla, blokace IP — provedeno podle definovaných playbooků.
- Integrace threat-intel feedů (MITRE ATT&CK, MISP) pro kontext detekcí.

Plán reakce na incidenty (IRP)

- Fáze: **příprava** → **identifikace** → **zadržení** → **eradikace** → **obnova** → **poučení**.
- **Red Team vs. Blue Team** cvičení; **Purple Team** sdílí znalosti a iteruje obranné mechanismy.
- Forenzní analýza a reporting pro vedení i regulátory.

7.8 Souvislý cyklus zlepšování

- **PDCA (Plan-Do-Check-Act)** – pravidelná aktualizace politik, technologií a školení.
- **Threat Hunting** – proaktivní vyhledávání anomálií v datech SIEM.
- **Lessons Learned** – po každém incidentu aktualizovat postupy a architekturu.

8. Technická a organizační opatření pro bezpečnost

Pro zajištění důvěrnosti, integrity a dostupnosti informací je nutné kombinovat technická a organizační opatření v souladu s požadavky ZoKB (zákon o kybernetické bezpečnosti), GDPR a směrnice NIS2. Technologie sama o sobě nestačí — bez vyškolených lidí a jasných procesů selhává i nejdražší řešení.

8.1 Technická opatření

1. Šifrování, pseudonymizace a anonymizace

- Symetrické šifrování (AES-256) a TLS 1.3 pro data v přenosu.
- Šifrování disků (BitLocker, LUKS) a databází (TDE — Transparent Data Encryption).
- Tokenizace, hashing a saltování pro pseudonymizaci osobních údajů. GDPR výslovně uvádí pseudonymizaci jako vhodné technické opatření.

2. Kontrola a správa přístupu

- MFA (TOTP, FIDO2) a silná heslová politika.
- PAM (Privileged Access Management) pro administrátorské účty.
- MDM (Mobile Device Management) + šifrování přenosných médií, možnost vzdáleného vymazání ztracených zařízení.
- Princip **nejmenších oprávnění** a pravidelné revokace přístupů.

3. Monitorování a detekce hrozeb

- SIEM s integrací threat-intel feedů a korelací logů v reálném čase.
- Continuous Vulnerability Management a automatický patch management.
- EDR/XDR na koncových bodech, sandboxing podezřelých souborů.

4. Zálohování a obnova dat

- **3-2-1 strategie** – 3 kopie dat, na 2 různých médiích, 1 mimo lokalitu.
- Offline / immutable zálohy (air-gap, WORM) odolné proti ransomwaru.
- Pravidelné testy obnovy a definovaný **Disaster Recovery plán**.

5. Bezpečnost síťové infrastruktury

- NGFW, IDS/IPS, WAF a kontrola „east-west“ provozu (mikrosegmentace, ZTNA).
- VPN (IPsec/SSL) a TLS inspection pro zabezpečené tunely.

6. Aplikační bezpečnost

- Secure SDLC: SAST, DAST, IAST, code-review, DevSecOps.
- RASP, bezpečné HTTP hlavičky (CSP, HSTS), ochrana API (rate-limiting, OAuth2).

7. Fyzická bezpečnost

- Kontrola vstupu (karty, biometrie), CCTV, redundantní napájení a chlazení.
- Detekce požáru, systémy pro potlačení kouře / inertní plyn (FM-200, Novec).

8.2 Organizační opatření

1. Bezpečnostní politika a řízení rizik

- Politika Data Classification & Handling, Data Retention & Secure Disposal.
- Pravidelná analýza rizik; opatření úměrná dopadu a pravděpodobnosti.
- ZoKB ukládá povinnost mít zavedeny tyto dokumenty a postupy.

2. Soulad s legislativou a standardy

- GDPR, ISO 27001, NIS2; vymezení odpovědností a pověřenec pro ochranu osobních údajů (DPO).
- **RoPA (Record of Processing Activities)**, smluvní zajištění třetích stran (DPA, SCC).

3. Školení a zvyšování povědomí

- Pravidelná školení (security awareness), phishing simulace, politika práce na dálku.
- Aktualizace materiálů při změně hrozeb či technologií.

4. Audit, testování a ověřování

- Interní a externí bezpečnostní audit, penetrační testy, vulnerability scanning.
- **DPIA (Data Protection Impact Assessment)** před novým zpracováním osobních údajů s vysokým rizikem.
- Pravidelné hodnocení účinnosti opatření a získávání certifikací (ISO/IEC 27001, ISO/IEC 27701).

5. Incident Response a Business Continuity

- Incident Response Plan: identifikace → zadržení → eradikace → obnova → poučení.
- Vazba na BCP (Business Continuity Plan) a Disaster Recovery; table-top, red/blue/purple-team cvičení.
- Hlášení incidentů NÚKIB (do 24/72 h) a porušení osobních údajů ÚOOÚ (do 72 h, pokud představují riziko pro subjekty).

6. Řízení přístupů a změn

- Princip **Least Privilege**, RBAC/ABAC, Segregation of Duties (SoD).
- Formální change-management proces s review a rollback plány.

Privacy by Design

GDPR vyžaduje koncept **Privacy by Design a by Default** — ochranu dat už od fáze návrhu systému a výchozí nastavení co nejvíce omezující okruh zpracování. Při plánování IT systémů je nutné od počátku zvažovat minimalizaci dat, anonymizaci pro testovací prostředí a defaultně omezené přístupy.

9. Zabezpečení vnějšího perimetru a interních systémů

Ochrana vnější hranice sítě se od ochrany vnitřních systémů liší cílem i nástroji. Vnější obrana je primárně preventivní — udržet útočníka venku. Vnitřní obrana je hloubková a detekční — předpokládá průnik a minimalizuje škody.

9.1 Zabezpečení vnějšího perimetru

Cílem je ochránit organizaci před útoky zvenčí — z internetu a dalších veřejných sítí. Hlavní opatření:

1. **Firewall** – řízení síťového provozu mezi interní a externí sítí podle pravidel (porty, IP adresy, stavy spojení). Princip *implicit deny*: vše nepovolené je blokováno.
2. **IDS/IPS** – detekce a případná blokace útoků v síťovém provozu. Bývají umístěné na hranici sítě.
3. **VPN** – šifrované spojení (IPsec, SSL/TLS) pro bezpečný vzdálený přístup zaměstnanců a partnerů.
4. **Ochrana proti DDoS** – cloudová scrubber centra (Cloudflare, Akamai) nebo lokální appliance, které absorbují masivní zahlcující útoky dříve, než dosáhnou infrastruktury.
5. **DNS bezpečnost** – ochrana proti DNS spoofingu, tunelování a přístupu na škodlivé domény (DNS filtering, DNSSEC).
6. **Bezpečnost e-mailů** – SPF, DKIM, DMARC, antiphishingové filtry, sandboxing příloh.
7. **WAF (Web Application Firewall)** – ochrana webových aplikací proti SQL injection, XSS, CSRF a dalším OWASP Top 10 hrozbám.
8. **Bezpečné API brány** – autentizace (OAuth2), šifrování, rate-limiting a ochrana před zneužitím.
9. **DMZ** – oddělená zóna pro veřejně přístupné servery (web, mail, VPN gateway), izolovaná od interní sítě.

9.2 Zabezpečení interních systémů (vnější i vnitřní hrozby)

Vnitřní síť vyžaduje jiný přístup — předpokládá se základní důvěra, ale insider hrozby ani úniky pomocí phishingu/malwaru nelze podceňovat. Statistiky ukazují, že významná část incidentů vzniká uvnitř.

1. **Segmentace sítě** – oddělení různých částí sítě (kanceláře, servery, výrobní technologie OT, hostovská Wi-Fi, citlivé databáze). Mezi segmenty platí pravidla interních firewallů.
2. **Zero Trust Architecture (ZTA)** – „nikomu nedůvěřuj, vždy ověřuj“. Každý přístup se autorizuje bez ohledu na polohu uživatele v síti.
3. **Ochrana koncových zařízení (EDR/XDR)** – antiviry, pokročilá detekce a reakce, izolace napadeného zařízení.
4. **Šifrování dat** – v klidu (disky, databáze) i během přenosu (TLS pro vnitřní komunikaci).
5. **Monitoring a logování (SIEM)** – centrální sběr a analýza událostí, detekce incidentů a anomálií.

6. **Přístupová kontrola** – princip nejmenších oprávnění, RBAC, MFA, auditní záznamy.

7. **Bezpečnostní aktualizace** – pravidelné záplatování OS, aplikací i firmware (správa zranitelností).

8. **Ochrana proti insider threats** – detekce podezřelého chování, omezení privilegií, čtyřočí princip u citlivých činností, DLP.

9.3 Rozdíl mezi obranou před vnějšími a vnitřními hrozbami

	Vnější hrozby	Vnitřní hrozby
Zdroj	Internet, hackeři, automatizované skenery, malware	Insideři, kompromitované stanice, uniklé přihlašovací údaje
Cíl obrany	Zabránit průniku do sítě (preventivní)	Detekovat průnik a omezit jeho dopad (detekční)
Klíčové nástroje	Firewall, IPS, WAF, DDoS ochrana, VPN	Segmentace, EDR, SIEM, monitoring chování
Filozofie	Tvrdá skořápka — implicit deny	Zero Trust — ověřuj každý přístup
Hloubka	Perimetrická (jedna pevná bariéra)	Vrstvená (více překážek po celé síti)

Souhrn

Silný perimetr bez vnitřní kontroly je nebezpečný v případě selhání jediné bariéry; vnitřní bezpečnost bez dobrého perimetru by byla zbytečně zatěžována množstvím útoků z internetu. Efektivní zabezpečení vyžaduje **kombinaci obou** — tvrdou skořápku navenek a ostrážité oko uvnitř.

10. Útoky na webovou infrastrukturu, dopady a obrana

Webové aplikace, API a servery čelí pestré škále útoků — od injektážních (SQLi, XSS) přes logické (CSRF, prolomení autentizace) až po útoky na dostupnost (DDoS). Tato kapitola popisuje hlavní hrozby, jejich dopady a vrstvená obranná opatření.

10.1 Hrozby

SQL Injection (SQLi)

- Útočník vkládá škodlivý SQL kód do vstupů aplikace, která ho neošetřeně použije v dotazu. Příklad: vložení ' OR '1'='1 do políčka hesla obejde autentizaci.
- **Dopady:** únik citlivých dat (uživatelé, hesla), modifikace nebo smazání dat, převzetí kontroly nad serverem (zvláště pokud má DB účet vysoká práva).

Cross-Site Scripting (XSS)

- Útočník vloží do stránky škodlivý skript (JavaScript), který se spustí v prohlížeči obětí. Typy: **reflektovaný** (skript v URL), **persistentní** (uložen v DB — např. v komentáři) a **DOM-based** (manipulace přímo v prohlížeči).
- **Dopady:** krádež cookies/session tokenů (převzetí účtu), logování stisknutých kláves, akce jménem uživatele, přesměrování na škodlivé stránky.

Cross-Site Request Forgery (CSRF)

- Útočník zneužívá důvěry aplikace v prohlížeč přihlášeného uživatele. Příklad: odkaz `banka.cz/prevod?ucet=123&castka=10000` spustí převod, protože prohlížeč automaticky pošle session cookie.
- **Dopady:** neautorizované akce (převody, změna hesla či e-mailu), závisí na funkcionalitě, kterou útočník zneužije.

DDoS (Distributed Denial of Service)

- Botnet zaplaví servery obrovským množstvím požadavků a způsobí přetížení (síťová linka, CPU, paměť, tabulka spojení).
- **Dopady:** nedostupnost služby, finanční ztráty (e-shop mimo provoz), poškození reputace, případně i pád systémů.

Man-in-the-Middle (MITM)

- Odposlech a manipulace nešifrované komunikace — útočník mezi klientem a serverem může číst data nebo je upravovat.

Brute-force a credential stuffing

- Hádání hesel (brute-force) nebo používání uniklých kombinací e-mail/heslo z jiných služeb (credential stuffing).

Path Traversal

- Přístup mimo webový adresář (např. `../../etc/passwd`) umožní čtení systémových souborů.

Zranitelnosti v API

- Nedostatečné ověření, IDOR (Insecure Direct Object Reference), chybějící rate-limiting nebo nesprávná autorizace.

Zero-Day útoky

- Zneužití dosud neopravené zranitelnosti, na kterou neexistuje záplata.

10.2 Důsledky útoků

- **Únik citlivých dat** – osobní údaje, přihlašovací údaje, firemní informace.
- **Finanční ztráty** – výpadky služeb, náklady na obnovu, regulatorní sankce (GDPR až 4 % obrátu).
- **Poškození reputace** – ztráta důvěry zákazníků a partnerů.
- **Právní dopady** – porušení GDPR, NIS2, oznamovací povinnost (72 h pro osobní údaje, 24/72 h pro NIS2 incidenty).
- **Malware infekce** – ransomware, backdoory, rootkity.

10.3 Ochranná opatření

Opatření	Jak chrání
WAF	Filtrování HTTP/HTTPS provozu, blokace SQLi, XSS, CSRF a aplikačních útoků.
TLS šifrování (HTTPS)	Ochrana proti MITM, šifrování komunikace, Secure cookie atribut.
Parametrizované dotazy	Prevence SQLi — oddělí kód od dat (prepared statements, ORM).
Escapování výstupů	Prevence XSS — zakódování HTML entit, použití frameworků s vestavěnou ochranou.
CSRF tokeny + SameSite cookies	Server ověří jednorázový token; SameSite zabrání odeslání cookie z cizího webu.
HttpOnly cookies	JavaScript nemůže číst session cookie — zmírnění dopadu XSS.
MFA a omezení pokusů	Vícefaktorová autentizace, CAPTCHA, lockout po N pokusech — proti brute-force.
Hashing hesel	Ukládat hesla jako BCrypt/Argon2 hashe se solí — ochrana při úniku DB.
DDoS ochrana	CDN, scrubber centra, rate-limiting, škálování serverů.
Bezpečné HTTP hlavičky	CSP, HSTS, X-Frame-Options, Referrer-Policy.
Patch management	Pravidelné aktualizace OS, frameworků a knihoven.
Monitoring (SIEM)	Detekce incidentů, korelace logů, alert při anomáliích.
Princip nejmenších oprávnění	Účet aplikace v DB má jen nezbytná práva, žádné DROP TABLE.
Penetrační testy a audit	Pravidelné odhalování slabých míst před útočníkem.
Bezpečný vývoj (DevSecOps)	SAST/DAST, code-review, školení vývojářů, OWASP zásady.

Klíčová zásada

Žádnou vrstvu nelze podcenit. SQLi a XSS se brání primárně na úrovni **aplikace (kód)**; CSRF kombinací **kódu a konfigurace**; DDoS zejména **infrastrukturně**. Útoky se neustále vyvíjejí, takže organizace musí sledovat aktuální bezpečnostní doporučení (OWASP Top 10), aktualizovat systémy a vzdělávat vývojáře i administrátory.